

Autonomous Defensive AI: An Envelope-and-Evidence Framework for Governed LLMs Acting on Production Infrastructure

Hiago Kin Levi
Decripte · hiago@decripte.us

This paper defines ADAI (Autonomous Defensive Artificial Intelligence), the proposed technical category for governed autonomous defense. It develops an action envelope, model-excluded admissibility, capability-disjoint verification, recomputable evidence, principal-scoped containment, reconciliation, and conditional constructions for composite actions and correlation.

Edition	Post-filing preprint prepared 1 October 2026
Manuscript	Technical body preserved from the supplied manuscript
Patent status	U.S. provisional application 64/166,855; patent pending
Publication status	Preprint; not represented as peer-reviewed
Language	English

The reported experiments and deployment observations refer to the version and period described in the manuscript. Formal results depend on their stated assumptions. This edition does not assert that every theoretical construction is implemented or independently validated, or that the software is publicly available.

© 2026 Hiago Kin Levi. All rights reserved. Public availability of this manuscript does not grant a license to proprietary source code, model weights, or patent rights. No third-party scientific endorsement is asserted.

Autonomous Defensive AI: An Envelope-and-Evidence Framework for Governed LLMs Acting on Production Infrastructure

Hiago Kin Levi

Post-filing edition. U.S. provisional application 64/166,855, received 1 October 2026. Patent pending; no patent has been granted. DAI is the proprietary LLM and its governed defensive architecture; ADAI names the proposed technical category.

Abstract

Defensive security automation is converging on a capability the field has not given a principled account of: software agents that decide, unprompted, to change the configuration of production infrastructure in order to stop an attack in progress. Existing work treats this either as orchestration (playbook engines, which execute but do not decide) or as control (reinforcement learning in simulated networks, which decide but do not act on production). Neither answers the question that determines deployability: under what conditions is it acceptable for a machine to act on infrastructure with no human in the loop, and what must it be able to prove afterwards?

We argue the answer is not a confidence threshold on a model's output but a structural property of the system. We define the **action envelope** as a conjunction of four independently checkable predicates — defended perimeter, bounded effect scope, reversibility with pre-declared rollback, and verification by a capability-disjoint credential — adjudicated by a decision function that is pure in a declared action contract and therefore provably independent of any learned component. We define the evidence obligation as a hash-linked, seal-terminated transcript with a **self-describing recomputation contract**, and prove that tamper localization is exact — recovering the altered index from the least failing index together with which verification conjunct failed, a distinction a boolean verifier discards, misattributing half of all single-turn alterations to the following, unaltered turn.

We identify and formalize two problems that we believe are new to the literature. The **discriminator condition** states necessary and sufficient conditions under which per-principal containment is expressible at a shared enforcement point, and shows that in the common case of multiple principals behind one enforcement domain, source-address predicates alone cannot achieve a blast radius of one. The **aggregation-reconciliation tension** shows that the rule-budget pressure which forces effects to be aggregated destroys the bijection between effect and rule that reconciliation algorithms assume, changing reconciliation from identifier-keyed lookup in $O(R)$ to element-wise existence testing in $O(A)$; we give the corrected algorithm, and show that the same aggregation makes the rule shared mutable state, admitting a lost update that the write reports as success and that only independent verification detects.

We give a five-level autonomy lattice ordered by *what may change without a human* rather than by model quality, an evaluation methodology separating three families of measurement whose conflation we argue is the field's central methodological error, and a characterization of the properties that admit no finite test and must therefore be discharged by construction.

We then resolve four difficulties the framework raises. The **atomization condition** states exactly when reconciliation from world to records is possible, and prescribes refusing the capability where it is not. A **blinded correlation index** yields cross-perimeter campaign detection without cross-perimeter disclosure, with its guarantee bounded precisely rather than asserted. A corrected accounting of corpus composition reduces "scarcity" to a class-imbalance question, which the approval gate then bounds one-sidedly. And **composite reversibility** across enforcement domains holds exactly when the declared effect scopes are pairwise disjoint — a condition checkable before execution, which turns an unbounded composition problem into one decision rule.

Keywords: autonomous cyber defense, agentic systems, action envelope, separation of duties, tamper-evident audit, tenant isolation, verifiable receipts, incident response.

1. Introduction

1.1 Three literatures that do not meet

Security orchestration. Playbook engines map a signal to a fixed response sequence [17, 4]. Every branch was authored by a person; when the signal matches no branch, nothing happens. Such systems are accountable precisely because they are not autonomous, and their accountability does not transfer to systems that are.

Autonomic computing. The Monitor–Analyse–Plan–Execute loop over a shared Knowledge base [18, 16] gave the field its vocabulary for self-managing systems. It is a control architecture and is silent on the question that dominates security: Plan may be wrong — adversarially so, since the input is partly authored by an adversary — and Execute changes a system other parties depend on. The loop describes the stages; it does not state what must hold before Execute may run.

Learned cyber defense. A substantial body of work trains agents, by reinforcement learning or by prompting large language models, to defend simulated networks [37, 14, 25]. These results are informative about policy quality and silent about deployment: simulators have no tenants, no credentials, no third parties whose traffic an erroneous block interrupts, and no obligation to explain afterwards.

The gap lies between the second and the third: an architecture for agents that decide, deployed where consequences are real.

1.2 ADAI: the class that falls in the gap

We name the class **autonomous defensive artificial intelligence (ADAI)** and define it by membership criteria rather than by capability, so that the question "is this an ADAI?" has an answer a reader can check rather than an answer its vendor asserts.

Definition 1.1 (ADAI). A system is an *autonomous defensive artificial intelligence* iff it satisfies all four of:

A. It acts on production infrastructure. It effects configuration changes to systems carrying real traffic for real dependents — not to a simulator, not to a staging copy, and not by producing a recommendation a person then enacts.

B. It decides without contemporaneous approval. For at least one class of action, the determination to act is the system's, taken at the time of the incident, with no human in that loop. A system whose every branch was authored in advance satisfies A and fails B.

C. Admissibility is adjudicated outside the component that proposes. The conditions under which action is permitted are evaluated by a function whose domain excludes the output of any learned or generative component, so that the permitted set is fixed before that component runs (§3).

D. It produces evidence a non-operator can verify. Each action yields a record whose integrity a party holding none of the system's code, and no access to its storage, can check from the record alone (§4).

Definition 1.2 (Degenerate cases). A system satisfying A and B but not C is *unconstrained*: its safety properties are properties of a component the adversary partly authors. A system satisfying A, B and C but not D is *unaccountable*: it may be safe and cannot demonstrate it, which is the same position as being untrusted. A system satisfying C and D but not B is an *audited orchestrator* — a valuable thing, and not this thing.

The criteria are deliberately independent, and each excludes a real class of system:

System	A	B	C	D	Class
Playbook orchestration (SOAR)	✓	✗	✓	partial	audited orchestrator
RL / LLM agents in simulated networks	✗	✓	✗	✗	not deployed
Agent gated on model confidence	✓	✓	✗	✗	unconstrained
Agent with internal audit log only	✓	✓	✓	✗	unaccountable
ADAI	✓	✓	✓	✓	—

1.2.1 The criteria as predicates

Let S be a system, $\text{Eff}(S)$ the set of configuration changes it effects, Dec_S its admissibility decision function, $\mathcal{M}_S$ the set of its learned or generative components, and $\text{Rec}_S(a)$ the record it emits for an action a .

$$\mathbf{A}(S) \equiv \exists a \in \text{Eff}(S) : \text{target}(a) \in \mathcal{W}_{\text{prod}}$$

$$\mathbf{B}(S) \equiv \exists \theta : \forall a \in \text{Eff}(S) \text{ of type } \theta, \text{exec}(a) \text{ requires no approval concurrent with } a$$

$$\mathbf{C}(S) \equiv \mathcal{M}_S \cap \text{dom}(\text{Dec}_S) = \emptyset$$

$$\mathbf{D}(S) \equiv \exists \text{Ver computable from } \text{Rec}_S(a) \text{ alone} : \text{Ver}(\text{Rec}_S(a)) \text{ decides integrity of } \text{Rec}_S(a)$$

and S is an ADAI iff $\mathbf{A}(S) \wedge \mathbf{B}(S) \wedge \mathbf{C}(S) \wedge \mathbf{D}(S)$.

Two clauses deserve emphasis. In **C**, the condition is on the *domain* of the decision function, not on its behaviour: it is not "the model rarely changes the outcome" but "the model is not an argument". In **D**, "computable from the record alone" excludes any verifier requiring the producer's code, the producer's storage, or a claim by the producer — which is what makes the criterion checkable by the party who has reason to doubt.

1.2.2 How to test membership

The criteria were chosen so that a reader can evaluate them without the vendor's cooperation on three of the four.

1. **A** — observe whether a configuration change occurred on a system carrying real traffic. Directly observable.
2. **B** — observe whether a human approved this instance. Ask for the approval record; absence of one for an action that occurred settles it.
3. **C** — read the signature of the admissibility function. If the vendor will not show it, the criterion is unsatisfied for the reader's purposes: an unexaminable claim is not a satisfied criterion. Where source is unavailable, a weaker but still useful test is adversarial: submit signals whose free-text fields argue for an inadmissible action and observe whether the permitted set moves.
4. **D** — take one record, implement a verifier from its published description, and run it. Then alter one field and confirm the verifier rejects, and that the index it reports is the index altered (§4.3). This is the strongest of the four tests and the cheapest to run.

A system failing test 4 is not an ADAI regardless of what its documentation says, and the test costs an afternoon.

Two consequences of stating the class this way are worth drawing out.

The criteria are falsifiable, and C and D are falsifiable by inspection. One checks C by reading the decision function's signature; one checks D by attempting the verification with none of the vendor's code and observing whether it succeeds. Neither requires trusting a claim. A category whose membership could only be asserted would not be worth naming.

Autonomy is not the achievement; C and D are. Criterion B is the easy one — any system can be made to act unprompted, and most of the hazard in this field comes from systems that have done exactly that. What makes the class coherent is that B is admitted *only* in the presence of C and D. An ADAI is therefore not "an AI that defends by itself"; it is an AI permitted to defend by itself because the conditions of its acting are fixed outside it and the record of its acting is checkable outside it.

The remainder of this paper gives the framework that discharges C and D. The companion paper describes a deployed system satisfying all four.

1.3 Why model confidence is the wrong gate

The instinct is to gate autonomous action on the model's confidence. This is the wrong instrument for three reasons, in increasing order of severity.

Calibration. Modern neural networks are poorly calibrated even in benign conditions [12, 27], and the input to a defensive agent is partly authored by the adversary [2, 11]. Any confidence derived from that input inherits the adversary's influence over it. Calibration guarantees obtained on benign distributions do not transfer.

Wrong codomain. Confidence is a scalar over the space of *propositions*. What governs admissibility is the space of *consequences*: how much is damaged if the proposition is false, and whether the damage is reversible. A high-confidence decision to null-route a /16 is strictly worse than a low-confidence decision to present a challenge to one address.

Non-persistence. A confidence threshold is a property of a component. Components are replaced, retrained, and reprompted. A guarantee located in a model's output does not survive any of those events; a guarantee located in the system's structure survives all three.

We therefore place the decision outside the learned component entirely, and the remainder of this paper makes that placement precise.

1.4 Contributions

1. A formal definition of the **action envelope** (§3) and a proof that the executable set is independent of any learned component (Prop. 3.4), with the consequent worst-case bound on blast radius (Cor. 3.4a).
2. A formal account of the **evidence obligation** (§4), with exact tamper localization (Thm. 4.3) and a *recomputation contract* making third-party verification possible without the producer's code (§4.4); and the observation (Prop. 4.8) that a chain of this shape authenticates content and position but **not principal attribution**, which in a multi-tenant deployment leaves the system's strongest isolation claim outside the artifact that is supposed to evidence it.
3. The **discriminator condition** (§5, Thm. 5.5): necessary and sufficient conditions for per-principal containment at a shared enforcement point; and a **granularity analysis** (Props. 5.9–5.11) showing that the budget-minimal layout is a single shared rule, that choosing it costs a q -fold increase in both the blast radius of a programming defect and the contention domain, and that no layout achieves credential-level isolation at enforcement points that scope tokens per domain.
4. The **aggregation–reconciliation tension** (§6, Prop. 6.3) and a corrected reconciliation algorithm with its complexity (Alg. 2); and its twin, the **lost-update consequence** of the same aggregation (§6.4, Prop. 6.7), together with the observation that the envelope's verification predicate detects it without having been designed for it.
5. An **autonomy lattice** ordered by admissible change rather than model quality (§7), and a **three-family evaluation methodology** (§8) with a characterization of the properties that admit no finite test (§8.4).
6. Resolutions (§11) of every difficulty the framework raises: the **atomization condition** (Thm. 11.2) stating exactly when world-to-records reconciliation is possible, with a **write-ahead intent log** (Constr. 11.10) for the enforcement points where it is not, converting an undetectable orphan into a detectable ambiguity the system is obliged to report (Cor. 11.12); a **blinded correlation index** (Constr. 11.4) giving cross-perimeter campaign detection without cross-perimeter disclosure, with its guarantee bounded precisely (Prop. 11.5); a corrected accounting of corpus composition (§11.3) together with a one-sided bound on the class-imbalance risk that follows from the narrator not deciding (Prop. 11.8); and **composite reversibility** across enforcement domains (Thm. 11.6) with the refusal condition it implies (Cor. 11.7).

2. Model and notation

2.0 Objects

Let P be a finite set of **principals** (tenants). Let $\mathcal{W}$ denote the configuration space of the defended infrastructure.

An **enforcement point** is a location at which configuration changes take effect. $\mathcal{G}$ denotes the set of **rule predicates** the enforcement point can express: for $g \in \mathcal{G}$, $\text{match}(g) \subseteq \mathcal{T}$ is the set of traffic the rule matches, where $\mathcal{T}$ is the space of requests.

For $p \in P$, $\text{traffic}(p) \subseteq \mathcal{T}$ is the traffic destined for p 's service. We assume $\{\text{traffic}(p)\}_{p \in P}$ partitions the traffic reaching the enforcement point.

The **perimeter map** $\Pi : P \rightarrow 2^{\mathcal{W}}$ assigns to each principal the configuration region it is entitled to modify.

An **action** is a tuple

$$a = \langle \theta, x, \pi, \sigma, \rho, \ell \rangle$$

where θ is the action type, x the target reference, π the parameters, $\sigma \subseteq \mathcal{W}$ the **declared effect scope**, ρ the **declared rollback** (a finite sequence of actions), and $\ell \in \mathcal{L}$ the required autonomy level (§7).

A **credential** c has capability set $\text{cap}(c) \subseteq \{\text{read}, \text{write}\} \times 2^{\mathcal{W}}$.

The **blast radius** of an action is

$$B(a) = |\{p \in P : \text{effect}(a) \cap \text{traffic}(p) \neq \emptyset\}|,$$

where $\text{effect}(a)$ is the traffic actually affected — which may differ from σ , and §3.2 is about forcing it not to.

2.1 Threat model

We state the adversary's capabilities explicitly, because every guarantee below is relative to them and several are void outside them.

The adversary may:

- T_1 — **author signal content**. Every field of an incoming signal, structured and free-text alike, may be chosen adversarially, including fields naming innocent third parties. This is not an assumption about a weak deployment; it is the defining condition of the problem, since signals describe the adversary's own traffic.
- T_2 — **choose timing and volume**. Signals may be injected in any pattern, including patterns designed to exhaust a rule budget (§5.3) or to force a review-cycle race.
- T_3 — **compromise a human operator account**. Testimony (§6.6) and, where enabled, class approvals at A_3 may originate from an attacker-controlled account.
- T_4 — **observe the system's outputs**, including narrations and receipts exposed to them.

The adversary may not:

- $\neg T_5$ — **obtain the executing credential** c_e . The envelope says nothing about an adversary who holds c_e ; such an adversary *is* the system's authority over the perimeter, and no property here survives.
- $\neg T_6$ — **write to the evidence store below the application**, nor to the seal D under its separate write authority (Cor. 4.5).
- $\neg T_7$ — **compute second preimages of H** .
- $\neg T_8$ — **control the enforcement point itself**. Verification (§3.1, E_4) reads the world through the same provider that executed the change. A provider that lies consistently to both credentials defeats E_4 , and the framework offers no defence; what E_4 excludes is the far more common case of the *system* being wrong about what it did.

What follows from the split. T_1 motivates Requirement 3.8 and is the reason the decision function excludes model output: an adversary with T_1 has partial authorship of any learned component's input, so a guarantee resting on that component's behaviour rests on the adversary. T_3 is the reason testimony is evidence and not command (§6.6): under T_3 , a command interface hands the perimeter to the attacker, whereas an evidence interface hands them at most an input to a review that still applies the envelope. $\neg T_8$ is the framework's most significant unmodelled assumption and we flag it as such: it is the single point whose compromise silently invalidates the verification guarantee, and it is shared with every system that verifies through the interface it acted on.

3. The action envelope

3.1 Definition

Definition 3.1 (Action envelope). For an action a proposed on behalf of principal p , under deployment policy Σ , the envelope predicate is

$$\mathcal{E}(a, p, \Sigma) = E_1 \wedge E_2 \wedge E_3 \wedge E_4$$

with

$$E_1 : \text{ep}(a) \in \Pi(p) \wedge \exists c_e : (\text{write}, \Pi(p)) \in \text{cap}(c_e) \wedge \forall (op, S) \in \text{cap}(c_e) : op = \text{write} \Rightarrow S \subseteq \Pi(p)$$

$$E_2 : \sigma \text{ is declared before execution} \wedge \sigma \subseteq \Sigma_{\max}(p, \theta) \wedge \text{effect}(a) \subseteq \sigma$$

$$E_3 : \rho \neq \varepsilon \wedge \text{rec}(\rho) \prec \text{exec}(a) \wedge \text{exec}(\rho) \circ \text{exec}(a) = \text{id} \upharpoonright_{\sigma}$$

$$E_4 : \exists c_v : (\text{read}, \sigma) \in \text{cap}(c_v) \wedge \forall (op, S) \in \text{cap}(c_v) : op \neq \text{write}$$

where $\text{ep}(a)$ is the enforcement point a writes to, $\text{rec}(\cdot)$ denotes durable recording, $\prec$ is the happens-before relation, and $\text{id} \upharpoonright_{\sigma}$ is the identity on the effect scope.

This is a conjunction, not a weighted score. There is no trade-off in which a very narrow σ compensates for $\rho = \varepsilon$.

3.2 The decision function and its independence

The separation of a *policy decision point* from a *policy enforcement point* is standard in access control [26, 33]; what follows makes the decision point's domain explicit, so that independence from a learned component is a property of the signature rather than of a runtime check.

Definition 3.2 (Decision function). The admissibility decision is

$$\delta : \mathcal{A} \times \mathcal{L} \times \{0, 1\} \longrightarrow \{\text{ALLOW}, \text{DENY}, \text{APPROVE}\},$$

$$\delta(a, \lambda, h) = \begin{cases} \text{DENY} & \text{if } \neg \mathcal{E}(a, p, \Sigma) \\ \text{DENY} & \text{if } \ell(a) \succ \lambda \\ \text{ALLOW} & \text{if } \ell(a) = \ell_{\max} \wedge \mathcal{E} \\ \text{APPROVE} & \text{if } \ell(a) \prec \ell_{\max} \wedge \neg h \\ \text{ALLOW} & \text{if } h \end{cases}$$

where λ is the deployment's autonomy ceiling and h indicates the presence of a recorded authorization emitted by a role holding authorize authority.

Crucially, h is an **input**, not an inference: δ does not derive approval from the contract's contents.

The naive independence claim — " δ is invariant in m " — is true and vacuous, since $m \notin \text{dom}(\delta)$ by construction. It is vacuous because a learned component still chooses *which* a is submitted, and different choices yield different decisions. The claim that carries weight concerns the **executable set**, not the decision on a fixed input.

Definition 3.3 (Executable set). For a learned component with output m , let $\mathcal{A}(m) \subseteq \mathcal{A}$ be the set of actions the system can construct under m , and

$$\text{Exec}(m, \lambda, h) = \{a \in \mathcal{A}(m) : \delta(a, \lambda, h) = \text{ALLOW}\}.$$

Proposition 3.4 (Independence of the executable set). Under the construction of Prop. 3.5, for every m ,

$$\text{Exec}(m, \lambda, h) \subseteq \text{Exec}^*(\lambda, h) := \{a \in \bigcup_{\theta} \text{Img}(\text{instantiate} \upharpoonright_{\text{cand}(\theta)}) : \delta(a, \lambda, h) = \text{ALLOW}\},$$

and Exec^* does not depend on m .

Proof. By Prop. 3.5, $\mathcal{A}(m) \subseteq \bigcup_{\theta} \text{Img}(\text{instantiate} \upharpoonright_{\text{cand}(\theta)})$ for every m , so the inclusion holds. Exec^* is defined from cand , instantiate and δ , none of which takes m as an argument; hence it is independent of m . $\square$

The distinction matters operationally. Prop. 3.4 does **not** say that a compromised model is harmless: within Exec^* it can choose the worst admissible action, and it can choose to act when not acting would be better. What it says is that the *boundary* of what can happen is fixed before the model runs, and is therefore auditable independently of it. An upper bound on damage that does not depend on the component most exposed to the adversary is the strongest property available here; a bound that *did* depend on it would be no bound at all.

Extend B to scopes by $B(S) = |\{p \in P : S \cap \text{traffic}(p) \neq \emptyset\}|$, which agrees with Def. §2 when $S = \text{effect}(a)$ and is monotone: $S \subseteq S' \Rightarrow B(S) \leq B(S')$.

Corollary 3.4a (Worst-case bound).

$$\max_m \max_{a \in \text{Exec}(m, \lambda, h)} B(a) \leq \max_{a \in \text{Exec}^*} B(a) \leq \max_{p, \theta} B(\Sigma_{\max}(p, \theta)).$$

Proof. The first inequality is Prop. 3.4 plus monotonicity of $\max$ over a subset. For the second, any $a \in \text{Exec}^*$ satisfies $\mathcal{E}$, hence E_2 , hence $\text{effect}(a) \subseteq \sigma \subseteq \Sigma_{\max}(p, \theta)$; monotonicity of B gives $B(a) = B(\text{effect}(a)) \leq B(\Sigma_{\max}(p, \theta))$. $\square$

The right-hand side is a deployment constant. Blast radius is therefore bounded by configuration, not by model behaviour — and the bound is *checkable without running the system*, by reading $\Sigma_{\max}$ and the traffic partition.

Proposition 3.5 (Catalogue closure). Let $\mathcal{C}$ be a finite catalogue of reviewed playbooks and let $\text{cand} : \Theta \rightarrow 2^{\mathcal{C}}$ map signal types to candidate sets. If action construction is defined as $a = \text{instantiate}(c, x)$ for $c \in \text{cand}(\theta)$, and a learned preference m may only select within $\text{cand}(\theta)$, then

$$\{a : a \text{ constructible}\} = \bigcup_{\theta} \{\text{instantiate}(c, x) : c \in \text{cand}(\theta)\},$$

independently of m .

Proof. The image of instantiate is determined by $\text{cand}(\theta)$, which is a fixed map. A preference is a choice function on $\text{cand}(\theta)$; choice functions do not enlarge their domain. $\square$

Corollary 3.6. A model may propose, rank, and narrate. It cannot widen the permitted set. Consequently, adversarial manipulation of the model's output — by prompt injection through the signal, by distribution shift, or by model substitution — cannot produce an action outside the catalogue or outside the envelope.

3.3 Target provenance

Definition 3.7 (Grounded target). A target x is *grounded* if it is extracted from the structured evidence field of the signal by a total parsing function $\text{ext} : \mathcal{J} \rightarrow 2^{\mathcal{X}}$ over a fixed key set, where $\mathcal{J}$ is the space of well-formed structured documents.

Requirement 3.8. Only grounded targets may be actioned. In particular, no target may be derived from natural-language fields.

The rationale is adversarial authorship, and it is the indirect prompt-injection threat [11, 30, 8] stated in terms of effects rather than of outputs. Free-text summaries are text an adversary influences; actioning an identifier appearing in one is a direct path to denial of service against a third party chosen by the adversary. Formally, if ext ranged over free text, then the adversary controls a subset of ext 's output, hence controls x , hence controls $\text{effect}(a)$ — while $\mathcal{E}$ still holds, because the envelope constrains *scope*, not *choice*. Requirement 3.8 is the complement the envelope does not provide.

Reviewed catalogues of adversary behaviour and of response procedures are the natural source for $\mathcal{C}$ and for cand [38, 4].

Definition 3.9 (Actionable kinds). Let K be the set of target kinds recognized by ext and $K_A \subseteq K$ those for which a reviewed playbook exists. Kinds in $K \setminus K_A$ are recorded as *observed* and not offered. The containment relation $K_A \subsetneq K$ is deliberate: an autonomous-system-number target, for instance, has blast radius equal to that provider's entire customer base, which no envelope predicate over a single principal can bound.

3.4 Why each predicate is structural, not procedural

Each predicate is chosen so that it can be discharged by construction rather than by convention, because convention fails silently.

E_1 is discharged by credential scoping and by binding the perimeter to the principal. The characteristic failure it prevents is subtle: a system that correctly filters *reads* by tenant and does not filter *actions*. Read isolation is cheap to audit — every query carries a visible filter, and every filter can be inspected — and its correctness gives a false impression of the whole. The action path is where isolation is expensive to get wrong and cheap to forget, because while a single tenant produces signal the omission is unreachable. §9 gives a measured instance, and the general shape is worth naming: *the auditable path and the consequential path are not the same path, and auditing effort flows to the former*.

E_2 is discharged by making effect scope a declared field of the contract, compared against a policy that is a property of the deployment rather than of the request. An agent able to describe its own action as narrow would be marking its own work. Note the asymmetry with E_1 : a perimeter violation is refused because the credential cannot express it, whereas a scope violation is refused because the declared value is compared against a constant the agent cannot write. The former is stronger; E_2 settles for the latter because effect scope is not a capability any enforcement point we know of exposes.

E_3 is discharged by requiring the rollback to be recorded before execution. Recording it afterwards is not equivalent, and the difference is exactly the failure window of §6.1: a system that dies between acting and recording leaves an effect in the world with no description of how to undo it. Ordering is the

guarantee, and it is free — the record is written anyway, and only its position relative to $\text{exec}(a)$ is at issue.

E_4 is **discharged by capability, not by code path**. "Whoever verifies did not execute" as a property of program structure is broken by a refactor, and broken *silently*: the verification still runs, still returns success, and now establishes nothing. As a property of the credential it survives refactoring — a defect that swapped the two objects would produce an authentication error on the write rather than a false verification. The verifying credential must therefore be shown, empirically and against the live interface, to be refused on write; a permission *name* suggesting read-only is a claim by the provider's documentation, not a measurement.

4. The evidence obligation

4.1 Why logging is insufficient

Logs answer *what does the system say happened*. That is the question one asks having already decided to trust the system. An autonomous defensive agent is deployed exactly where that trust is not yet established. The artifact must answer *what happened, and can I check it without trusting you?*

4.2 Construction

The construction is the classical hash-linked log [13, 21, 35, 6, 20], specialized so that the chaining field is *inside* the hashed object.

A session is a finite sequence of **turns** $T = \langle \tau_0, \dots, \tau_n \rangle$. Fix an ordered field set $F = \langle f_1, \dots, f_k \rangle$ and a canonical serialization κ (deterministic key order, fixed separators, fixed encoding, fixed non-serializable coercion). Let H be a collision-resistant hash.

$$\tau_i.\text{prev} = \begin{cases} \perp & i = 0 \\ d_{i-1} & i > 0 \end{cases} \quad d_i = H(\kappa(\tau_i)), \quad \text{prev} \in F.$$

Note that the chaining is *internal*: prev is one of the k hashed fields, so d_i depends on d_{i-1} through κ rather than through concatenation. This matters for the recomputation contract (§4.4), because a verifier must reconstruct the object, not merely concatenate.

The session record additionally stores a **seal** D , written at sealing time in a location outside the chain, with the integrity constraint that a session cannot be marked sealed while D is absent.

Definition 4.1 (Validity).

$$\text{Valid}(T, D) \equiv \left(\forall i \leq n : d_i = H(\kappa(\tau_i)) \right) \wedge \left(\forall i > 0 : \tau_i.\text{prev} = d_{i-1} \right) \wedge (\tau_0.\text{prev} = \perp) \wedge (d_n = D).$$

Requirement 4.2 (Append-only below the application). Insertion is the only admissible mutation, enforced at the storage layer. A system that *intends* not to rewrite history and one that *cannot* are different systems; only the latter survives a defect in the application.

4.3 Tamper localization is exact

Let the verifier report, for each i , which of two conjuncts fails:

$$C_i^{\text{dig}} \equiv d'_i = H(\kappa(\tau'_i)), \quad C_i^{\text{lnk}} \equiv \tau'_i.\text{prev} = d'_{i-1},$$

and let $v(T') = \min\{i : \neg C_i^{\text{dig}} \vee \neg C_i^{\text{lnk}}\}$.

Theorem 4.3 (Localization). Let T' differ from T exactly in the turns indexed by a non-empty J , with $j^* = \min J$, and suppose the adversary cannot compute second preimages of H . Then exactly one of:

- (a) the adversary left $d'_{j^*} = d_{j^*}$; then $v(T') = j^*$ and the failing conjunct is C^{dig} ;
- (b) $j^* < n$, the adversary recomputed $d'_{j^*} = H(\kappa(\tau'_{j^*}))$ and did not recompute the suffix; then $v(T') = j^* + 1$ and the failing conjunct is C^{lnk} ;
- (c) the adversary recomputed d'_i for all $i \geq j^*$ — which includes the case $j^* = n$ with d'_n recomputed, since the suffix is then empty; then every C_i^{dig} and C_i^{lnk} holds and $d'_n \neq d_n = D$, so validity fails at the seal, whose repair requires a second write authority.

Proof. For $i < j^*$ no turn differs and both conjuncts hold, so $v(T') \geq j^*$. In case (a), $C_{j^*}^{\text{dig}}$ asserts $d_{j^*} = H(\kappa(\tau'_{j^*}))$ with $\tau'_{j^*} \neq \tau_{j^*}$, a second preimage of H under d_{j^*} ; it therefore fails except with probability negligible in the security parameter. In case (b), $C_{j^*}^{\text{dig}}$ holds by construction and $C_{j^*}^{\text{lnk}}$ holds because $\tau'_{j^*}.\text{prev} = d_{j^*-1}$ is unchanged; at $i = j^* + 1$ the turn is unaltered, so $C_{j^*+1}^{\text{dig}}$ holds, while $\tau_{j^*+1}.\text{prev} = d_{j^*} \neq d'_{j^*}$ falsifies $C_{j^*+1}^{\text{lnk}}$. Case (c) is (b) applied inductively to n ; the terminal digest then differs from the independently stored D . $\square$

Corollary 4.4 (Exact localization). Define

$$\hat{j}(T') = \begin{cases} v(T') & \text{if } \neg C_{v(T')}^{\text{dig}} \\ v(T') - 1 & \text{if } C_{v(T')}^{\text{dig}} \wedge \neg C_{v(T')}^{\text{lnk}} \end{cases}$$

Then $\hat{j}(T') = j^*$ in cases (a) and (b). Case (c) yields no failing index at all — no $\hat{j}$ is defined — and is detected only at the seal, which is why the seal must not share a write authority with the chain.

The refinement is not cosmetic. Reporting only the least failing index misattributes case (b) to turn $j^* + 1$ — an unaltered turn — which in an investigation points at the wrong actor, the wrong moment, and the wrong action. Which conjunct failed is the information that distinguishes the two, and a verifier that reports only a boolean discards it.

This is the property that distinguishes a chain from a checksum, and it yields a directly measurable quantity: **tamper-localization accuracy**, the fraction of synthetically altered chains for which $\hat{j}(T') = j^*$ (§8.3).

Corollary 4.5 (Forgery cost). By Thm. 4.3, forging a session requires either a second preimage of H — of probability negligible in the security parameter — or case (c): rewriting the entire suffix *and* the out-of-chain seal. Placing the seal under a distinct write authority from the transcript therefore raises forgery from one compromise to two.

4.4 The recomputation contract

Tamper evidence is worthless if only the producer can evaluate it.

Definition 4.6 (Recomputation contract). An exported receipt satisfies the recomputation contract iff it carries: (i) the hash function; (ii) the ordered field list F ; (iii) the full specification of κ ; (iv) the chaining rule; (v) both D and the recomputed d_n , separately, so divergence is visible; and (vi) for every turn, **all** k fields of F .

A receipt satisfying (i)–(vi) authenticates exactly what F covers. Whatever F omits is asserted by the receipt and verified by nothing — see Prop. 4.8.

Clause (vi) is the one that fails in practice. A receipt that exports a *projection* of the turn — a subset chosen for display — cannot be recomputed by anyone, and the failure is invisible to every test written against the producer's own serialization, because such tests never reconstruct the object from the receipt.

Requirement 4.7 (Independent verification). Conformance is demonstrated by a verifier implemented from the receipt's published description alone, importing none of the producing system's code. Any weaker demonstration establishes only that the producer agrees with itself.

4.5 What the chain proves

The distinction is the one the provenance literature draws between the integrity of a record and the veracity of what it records [24].

It establishes **process**: that these turns, in this order, with these contents, were recorded and not subsequently altered; that the contract preceded execution; that verification was recorded under a role distinct from the executor.

It does not establish **origin**: that the signal was true, that the adversary was who the evidence names, or that the world matched the description. Conflating the two is the most attractive error available to such a system, and the mitigation is that the receipt state its own limits as part of the artifact.

It also does not establish attribution, and this is easy to miss. The field set F covers the turn's content, its role and authority, its position, and its link to the predecessor. It does not cover the *principal* the session belongs to. The turn carries the session identifier and the session record carries the principal, so attribution lies one dereference outside the chain.

Proposition 4.8 (Attribution is unauthenticated). Let $\text{owner} : \mathcal{S} \rightarrow P$ be the session-to-principal map, stored outside T . Altering $\text{owner}(s)$ leaves $\text{Valid}(T, D)$ unchanged.

Proof. owner appears in no $f \in F$, so $\kappa(\tau_i)$, hence every d_i , hence $D = d_n$, is invariant in it. $\square$

The consequence is precise and worth stating plainly rather than leaving implicit: a third-party verifier following Req. 4.7 can confirm *what happened*, and cannot confirm *on whose behalf*. In a multi-tenant deployment — where the principal is the subject of the strongest isolation guarantee the system makes (§5) — that is the wrong thing to leave outside the authenticated set. In practice the binding is enforced by an append-only constraint on the store, which is a real guarantee but a *local* one: it protects the operator's own records and is worth nothing to an external reader, who has no access to the store and no way to observe that the constraint exists.

Two remedies exist and they are not equivalent. Adding the principal to F authenticates attribution cryptographically, at the cost of recomputing every historical digest — which an append-only store exists to prevent, making it a fork of the chain rather than an edit to it. Binding the session record into turn 0's payload authenticates attribution for sessions opened after the change, leaves history alone, and is therefore what a deployed system can actually do. We state both, because a paper offering only the clean remedy describes a system nobody can migrate to.

5. The discriminator condition

This section addresses a problem we have not found treated in the literature: when multiple principals share one enforcement point, under what conditions is per-principal containment expressible at all?

5.1 Setting

Isolation in multi-tenant systems is usually studied on the data and storage planes [19]. The object here is different: the shared resource is the *enforcement point*, and what must be isolated is the effect of writing to it.

Let Z be an enforcement domain (for example, a DNS zone fronted by one edge network) serving principals $P_Z \subseteq P$ with $|P_Z| = q > 1$. Let $\mathcal{G}$ be the expressible rule predicates.

Definition 5.1 (Tenant-sound containment). A rule $g \in \mathcal{G}$ containing target x on behalf of p is *tenant-sound* iff

$$\text{match}(g) \subseteq \text{traffic}(p) \quad \text{and} \quad \text{from}(x) \cap \text{traffic}(p) \subseteq \text{match}(g),$$

i.e. it contains all of x 's traffic to p and none of anyone else's.

Definition 5.2 (Blast radius one). $B(g) = 1$.

Lemma 5.3. If every $g \in \mathcal{G}$ has the form $\text{from}(x)$ — a predicate over source alone — then for any x with traffic to more than one principal, $B(g) = |\{p : \text{from}(x) \cap \text{traffic}(p) \neq \emptyset\}| > 1$, and no g is tenant-sound.

Proof. $\text{match}(g) = \text{from}(x)$ is independent of p and intersects every principal's traffic that x reaches. The first clause of Def. 5.1 fails. $\square$

This is the formal content of a failure that is easy to miss operationally: a source-address block at a shared edge, raised because of principal p_1 's incident, silently denies service to $p_2, \dots, p_q$ — including, in the deployment of §9, the very console an operator would use to observe the incident.

5.2 The condition

Definition 5.4 (Discriminator). A function $d : \mathcal{T} \rightarrow \mathcal{D}$ is a *discriminator* for P_Z iff there exists $\phi : P_Z \rightarrow 2^{\mathcal{D}}$ with $\{\phi(p)\}_{p \in P_Z}$ pairwise disjoint and

$$\forall p \in P_Z, \forall t \in \mathcal{T} : \quad t \in \text{traffic}(p) \iff d(t) \in \phi(p).$$

We assume, as holds at every enforcement point we are aware of, that $\mathcal{G}$ is closed under conjunction and under finite disjunction, and that the source space $\mathcal{X}$ is finite.

Theorem 5.5 (Discriminator condition). The following are equivalent:

1. Tenant-sound containment is expressible at Z for every $p \in P_Z$ and every $x \in \mathcal{X}$.
2. For every $p \in P_Z$ there exists a **separating predicate** $\pi_p \in \mathcal{G}$ with $\text{match}(\pi_p) = \text{traffic}(p)$.
3. There exists a discriminator d for P_Z whose classes are expressible in $\mathcal{G}$, and containment takes the form

$$g_{p,x} = (d(t) \in \phi(p)) \wedge (t \in \text{from}(x)).$$

Proof. (2 $\Rightarrow$ 1) Take $g_{p,x} = \pi_p \wedge \text{from}(x)$, which is in $\mathcal{G}$ by closure under conjunction. Then $\text{match}(g_{p,x}) = \text{traffic}(p) \cap \text{from}(x)$, satisfying both clauses of Def. 5.1; and since $\{\text{traffic}(p)\}$ partitions, $B(g_{p,x}) = 1$.

(1 $\Rightarrow$ 2) Fix p and let $g_{p,x}$ be tenant-sound for each x . Put $\pi_p = \bigvee_{x \in \mathcal{X}} g_{p,x}$, which is in $\mathcal{G}$ by closure under finite disjunction. The first clause of Def. 5.1 gives $\text{match}(g_{p,x}) \subseteq \text{traffic}(p)$ for each x , hence $\text{match}(\pi_p) \subseteq \text{traffic}(p)$. Conversely, let $t \in \text{traffic}(p)$ and let x be its source; the second clause gives $\text{from}(x) \cap \text{traffic}(p) \subseteq \text{match}(g_{p,x})$, and t belongs to the left side, so $t \in \text{match}(\pi_p)$. Hence $\text{match}(\pi_p) = \text{traffic}(p)$.

(2 $\Leftrightarrow$ 3) Given $\{\pi_p\}$, set $\mathcal{D} = P_Z$, $d(t) =$ the unique p with $t \in \text{match}(\pi_p)$ — well defined because $\{\text{traffic}(p)\}$ partitions — and $\phi(p) = \{p\}$; Def. 5.4 holds, and the class $\{p\}$ is expressed by π_p . Conversely, given a discriminator whose classes are expressible, $\pi_p := (d(t) \in \phi(p))$ has $\text{match}(\pi_p) = \text{traffic}(p)$ by Def. 5.4. $\square$

The equivalence of (1) and (2) is what makes the condition useful in practice: one need not search for a discriminator, only ask whether the enforcement point can express *one predicate per principal that matches exactly that principal's traffic*. If it cannot, no amount of programming in the agent will produce tenant-sound containment, and the correct engineering response is to change the enforcement point rather than the agent.

Corollary 5.6. When principals are separated by service hostname, the host field is a discriminator, and tenant-sound containment requires the enforcement point to express predicates conjoining host and source. Enforcement points offering only source-based access rules are insufficient regardless of how the system is programmed; the limitation is in $\mathcal{G}$, not in the agent.

Corollary 5.7 (Undefendable principals). If p 's traffic does not traverse Z , then no $g \in \mathcal{G}$ affects it, and registering p as defended at Z causes the system to record containments that do not exist. The correct behaviour is to refuse registration: an unrecorded false negative is strictly preferable to a recorded false positive about the state of the world.

5.3 Rule-budget pressure

Real enforcement points bound $|\mathcal{G}|$ in use. Let L be the platform's limit on custom rules and k the number already deployed for unrelated purposes, giving budget $L - k$.

Write $\mathcal{X}_p$ for the set of addresses currently contained for p and $N = \sum_p |\mathcal{X}_p|$ for the total number of containments in force. Three granularities are expressible, and we give the rule count each requires.

Definition 5.8 (Granularity). A *containment layout* is a function assigning each containment (p, x) to a rule. Its *cost* $\mathcal{R}$ is the number of distinct rules used.

- **Per containment:** $g_{p,x} = (d(t) \in \phi(p)) \wedge (\text{src}(t) = x)$, so $\mathcal{R} = N$.
- **Per principal:** $g_p = (d(t) \in \phi(p)) \wedge (\text{src}(t) \in \mathcal{X}_p)$, so $\mathcal{R} = |\{p : \mathcal{X}_p \neq \emptyset\}| \leq q$.
- **Single shared rule:** $g^* = \bigvee_p [(d(t) \in \phi(p)) \wedge (\text{src}(t) \in \mathcal{X}_p)]$, so $\mathcal{R} = 1$.

Proposition 5.9 (Budget). All three layouts are tenant-sound (Def. 5.1), and $\mathcal{R}$ is N , at most q , and 1 respectively. Under budget $L - k$, the per-containment layout is feasible only while $N \leq L - k$; the other two are feasible for all N up to the expression-length limit.

Proof. Tenant-soundness: each disjunct of each layout has $\text{match} \subseteq \text{traffic}(p)$ by Thm. 5.5, and the disjunction in g^* is over pairwise-disjoint $\{\phi(p)\}$, so no cross-principal matching arises. Counts are immediate. $\square$

The budget-minimal layout is therefore the single shared rule, and it is not the one to choose. Stating why is the point of this subsection, because "we aggregated to fit the budget" would leave the reader with an argument that selects the wrong layout.

What granularity does not buy. It does *not* buy credential isolation. The enforcement points we surveyed scope write tokens to the **domain**, not to individual rules, so under every layout the executing credential for p is technically able to write any rule in Z . E_1 's guarantee therefore extends to *which domain* a

credential may write, and stops short of *which rule within it*. This is a real gap between the framework's ideal and what enforcement points currently expose, and we flag it rather than describe the layout as achieving something it does not.

What granularity does buy is two things, both about failure rather than authority.

Proposition 5.10 (Fault containment). Let a defect corrupt the expression written for principal p . Under the per-principal layout the corruption is confined to $\mathcal{X}_p$; under the single shared rule it may destroy $\bigcup_{p'} \mathcal{X}_{p'}$.

Proof. Writing g_p addresses a rule whose match set is contained in $\text{traffic}(p)$; no other principal's containment is represented in that object. Writing g^* addresses the single object representing every principal's containment. $\square$

Proposition 5.11 (Contention). Under the read-modify-write of §6.4, the per-principal layout admits a lost update only between two episodes of the *same* principal; the single shared rule admits one between *any* two episodes.

Proof. Contention is on the written object. Under the per-principal layout episodes of distinct principals write distinct objects; under g^* all episodes write one. $\square$

The choice. We take the per-principal layout: $\mathcal{R} \leq q$ rather than the minimum 1, paying $q - 1$ rules for a q -fold reduction in the blast radius of a defect and in the contention domain. This is the same exchange the framework makes everywhere — accept a bounded, stated cost to convert a correlated failure into an isolated one — and it is worth noting that the budget-minimal layout is also the one in which a single programming error silently disarms the entire installation.

The per-containment layout is excluded not by preference but by feasibility: for the deployment of §9, $L - k$ is a small integer, so $\mathcal{R} = N$ caps the installation at that many simultaneous containments. That is not a design constraint; it is a toy.

Aggregation is therefore forced by the budget, and the *granularity* of the aggregation is chosen against fault containment. With it, §6's problems appear — both of them, and they are consequences of the same decision.

6. The aggregation–reconciliation tension

6.1 Why reconciliation is mandatory

The world→records traversal is the convergence problem of declarative configuration management [3, 15] under a constraint those systems do not have: the record is evidential, so divergence may not be repaired by rewriting the record.

Post-action review traverses *records* → *world*: for each recorded containment, is it still warranted? This leaves a hole. Let t_{act} be the instant the effect takes hold and t_{rec} the instant the record is durable. These cannot be made simultaneous. An effect created in the window $[t_{\text{act}}, t_{\text{rec}})$ during which the process fails is **an effect in the world with no record**, and it is invisible to record-driven review forever.

Requirement 6.1. The system must also traverse *world* → *records*.

Definition 6.2 (Adoption). An unrecorded effect is *adopted*: the system opens a session recording what it observes **now**, explicitly declining to assert the moment of creation. Reconstructing a plausible creation timestamp would place in an append-only evidential record a proposition no one observed — corrupting the only property the record promises. The expiry clock starts at adoption, which errs toward retaining protection rather than removing it early.

6.2 The tension

Record-driven reconciliation assumes an identification map $\text{id} : \mathcal{R} \rightarrow \mathcal{S} \cup \{\perp\}$ from rules to sessions, recoverable from the rule itself (e.g. an identifier embedded in its description).

What reconciliation actually needs is not id but the *origin map* $\text{orig}_p : X_p \rightarrow \mathcal{S}$, giving the session that contained each address. The question is whether orig_p is recoverable from the rule.

Proposition 6.3 (Origin is not recoverable under aggregation). Suppose the rule's observable representation is $\langle \phi(p), X_p, \text{desc}(p) \rangle$, where desc depends only on p , and X_p is a *set* — that is, the representation is invariant under permutation of X_p and carries no per-element annotation. If $|X_p| \geq 2$, then no function of the rule computes orig_p .

Proof. Let $x_1 \neq x_2 \in X_p$ and let $s_1 \neq s_2$ be sessions. Consider two histories H_1, H_2 producing the same rule, with $\text{orig}_p^{(1)}(x_1) = s_1, \text{orig}_p^{(1)}(x_2) = s_2$ and $\text{orig}_p^{(2)}(x_1) = s_2, \text{orig}_p^{(2)}(x_2) = s_1$. Both yield the identical representation $\langle \phi(p), \{x_1, x_2\}, \text{desc}(p) \rangle$. A function of the rule takes the same value on both, yet $\text{orig}_p^{(1)} \neq \text{orig}_p^{(2)}$. $\square$

Corollary 6.3a. id is therefore not merely unavailable but ill-posed: any total $\text{id} : \mathcal{R} \rightarrow \mathcal{S}$ selects one of $|X_p|$ origins, and its choice is arbitrary. An implementation that embeds *one* session identifier in the description — a natural first attempt — is not a partial solution; it is a rule that attests to the provenance of one address and silently misattributes the rest.

Corollary 6.4. Reconciliation algorithms keyed on rule identity are *unsound* under aggregation, in both directions:

- **False negative:** an unrecorded address inside an otherwise-recorded rule is never detected, because the rule "has a session".
- **False positive:** if the recordedness test is evaluated per rule and fails for any reason, the entire rule — every legitimate containment in it — is re-adopted on every cycle, producing unbounded duplicate records.

Both failure modes were observed in the deployment of §9.

6.3 Corrected algorithm

The unit of reconciliation must be the unit of effect.

Algorithm 1 — Reconciliation, rule-identified (sound only without aggregation)

```
for g in rules_marked_as_ours(Z):
    s ← id(g)
    if s = ⊥ or not has_execution_record(s):
        adopt(g)
```

Cost: $O(R)$ lookups, R = number of rules.

Algorithm 2 — Reconciliation, element-identified (sound under aggregation)

```

for p in principals_defended_at(Z):
    X ← read_contained_set(c_v(p))      # verifying credential only
    if X = ⊥: continue                  # unreadable ≠ empty; defer
    for x in X:
        if not exists_record(p, x):
            adopt(p, x, mode ← read_mode(c_v(p), x))

```

Cost: $O(A)$ existence queries, $A = \sum_p |X_p|$.

Three details are load-bearing and each corresponds to an observed defect.

Read through the verifying credential. Reconciliation is observation; using the executing credential would permit a defect in the reconciler to modify the perimeter it is auditing.

Unreadable is not empty. If the enforcement point cannot be read, treating $X = \emptyset$ silently concludes "nothing is contained" and abandons the traversal; treating every address as orphaned adopts the world. The correct behaviour is to defer.

The adopted mode is read, not assumed. Recording a default severity when the actual one differs causes the next review to compare what it reads against what it recorded and conclude divergence where none exists.

6.4 Aggregation also makes the rule shared mutable state

The reconciliation problem of §6.2 is the better-known consequence of aggregation. There is a second, and it is a concurrency property rather than an identification one.

Definition 6.6 (Containment as read-modify-write). Under §5.3, containing x for principal p is the operation

$$\text{contain}(p, x) : \quad X_p \leftarrow \text{read}(g_p); \quad X'_p \leftarrow X_p \cup \{x\}; \quad \text{write}(g_p, X'_p),$$

with no atomicity between the read and the write unless the enforcement point offers a compare-and-swap precondition on g_p .

Proposition 6.7 (Lost update). Let two containments $\text{contain}(p, x_1)$ and $\text{contain}(p, x_2)$ interleave as

$$\text{read}_1 \prec \text{read}_2 \prec \text{write}_1 \prec \text{write}_2,$$

with both reads returning X_p . Then the final state is $X_p \cup \{x_2\}$, and x_1 is not contained although its episode executed, verified nothing to the contrary at write time, and would seal.

Proof. write_2 carries $X_p \cup \{x_2\}$, computed from read_2 , which preceded write_1 and therefore did not observe x_1 . The last write wins. $\square$

Three observations make this more than a textbook race.

It is not detected by the write. The provider returns success: the rule was updated, exactly as asked. The failure is that *what was asked* was computed from stale state.

It is detected by E_4 , and that is not an accident. Independent verification re-reads the world rather than trusting the write's response (§3.4), so the losing episode's verification returns **failed**. The envelope's fourth predicate, introduced to catch the system being wrong about what it did, catches this without knowing the mechanism. This is the kind of coverage a structural guarantee gives and a targeted check does not.

Detection is not recovery. A verified-failed episode seals honestly and leaves x_1 uncontained. The available responses are ordered by strength:

1. *Compare-and-swap on g_p* , where the enforcement point offers it. Correct, and unavailable at the per-rule endpoints we surveyed.
2. *Serialize writes per principal*, e.g. by a per- p lease. Correct, and adds a coordination dependency to the action path — the component whose availability matters most.
3. *Bounded re-read-and-retry*. Recovers the single-conflict case, which is the overwhelmingly common one, and degrades to (detected) failure beyond it.
4. *Depend on there being one writer*. This is the tempting one, and it is wrong in a specific way worth naming: single-writer is a property of the **deployment topology** — a process count, a queue consumed sequentially — and not of the component. It is true until someone scales out, and the scheduled review task, which also writes to the perimeter, already violates it.

We take (3), with a retry bound of one. A loop would mask sustained contention on a single principal, and sustained contention is a signal about the incident rate, not a condition to absorb silently.

Corollary 6.8. Aggregation trades a rule-budget constraint for two consequences: the origin map is lost (Prop. 6.3), and the rule becomes shared mutable state (Prop. 6.7). Both are consequences of the *same* decision, and a system that adopts aggregation for capacity reasons without addressing both has adopted half of it.

6.5 Review as a decision function

Definition 6.9. Let $\text{present} \in \{0, 1\}$ (rule observed via c_v), $\text{recur} \in \{0, 1\}$ (target recurred since the containment), Δt elapsed time, W the non-recurrence window, and $\text{test} \in \{\perp, \text{FP}, \text{CONF}, \text{CTX}\}$ recorded human testimony. The verdict function is

$$V = \begin{cases} \text{FALSE_POSITIVE} & \text{test} = \text{FP} \\ \text{ABSENT} & \neg \text{present} \\ \text{REAFFIRMED} & \text{present} \wedge \text{recur} \\ \text{TOO_EARLY} & \text{present} \wedge \neg \text{recur} \wedge \Delta t < W \\ \text{EXPIRED} & \text{present} \wedge \neg \text{recur} \wedge \Delta t \geq W \end{cases}$$

TOO_EARLY opens no session. "Come back later" is not an episode, and recording every non-decision trains the operator to stop reading the record — a degradation invisible to every technical metric and well documented in operations research on alarm fatigue [1, 7].

6.6 Testimony as evidence, never as command

Human knowledge must be admissible: an operator may know a contained address belongs to a partner. We admit it as a **recorded classification** entering the evidence stream, read by the next review, which may then reach **FALSE_POSITIVE** and undo through the ordinary action path.

We do **not** admit it as an instruction that actuates the perimeter. The distinction is a security property, not a stylistic one: an interface in which a person's assertion directly changes the perimeter is an interface in which compromising that person's account directly changes the perimeter. Routing testimony through

evidence preserves the envelope — the removal remains an action, contracted, verified, and recorded.

7. The autonomy lattice

Levels of automation have a long history in human-factors research [36, 29] and a widely-cited industrial instance in driving automation [31]. Those taxonomies are ordered by *who performs which function*. Ours is ordered by *what class of state may change without a human*, which is the ordering that matters when the automation's output is a configuration change to infrastructure other parties depend on.

Definition 7.1. $\mathcal{L} = \{A_0 \prec A_1 \prec A_2 \prec A_3 \prec A_4\}$, a total order, where the level of an action is determined by *what class of state it may change without human approval*:

Level	May change without a human	Human role
A_0	nothing	performs all action
A_1	nothing; proposes a specific action	approves each instance
A_2	own telemetry and internal state	approves external effects
A_3	defended perimeter, within $\mathcal{E}$	approves <i>classes</i> of action
A_4	defended perimeter, within $\mathcal{E}$	reviews after; may reverse

Two properties matter more than the table.

Proposition 7.2 (Orthogonality to model quality). The level required by an action is a function of the action's type and effect class alone: $\ell = \ell(\theta, \sigma)$. In particular, ℓ is independent of any measure of the proposing component's quality.

Consequence. Improving the model does not expand authority. This is a deliberate design choice with a real cost — a better model cannot be rewarded with more latitude — and a real benefit: authority changes only by explicit decision, recorded as such.

Definition 7.3 (Ceiling). The deployment fixes $\lambda \in \mathcal{L}$; actions with $\ell \succ \lambda$ are refused with a recorded reason. An agent cannot raise λ .

Requirement 7.4. Where the code carries a default λ_{def} and the deployment carries λ_{env} , the deployment must assert λ_{env} on every rollout. Otherwise the two drift, and the code's default silently governs exactly those deployments nobody re-checked.

7.1 A_4 is not the absence of a human

A_4 removes the human from the *decision*, not from the *loop*. The human's role relocates to three positions, all of which must exist:

1. **Before:** fixing Σ , Π , and λ .
2. **After:** reviewing from a record that supports review (§4).
3. **Against:** the standing ability to reverse any action, and to enter testimony the system treats as evidence (§6.6).

A system at A_4 lacking (2) and (3) is not autonomous; it is unsupervised, which is a different and worse property.

7.2 The twelve mechanisms, named

§§3–7 define twelve mechanisms. Referring to each by its defining number is precise and unusable in conversation, in a code review, or in a specification that a reader traverses non-linearly. We therefore name them.

The names are bound to the formal results, not parallel to them. Each is an abbreviation for a definition or theorem already stated; none introduces meaning the mathematics does not carry. A reader who distrusts a name can discard it and lose nothing but brevity.

Name	Abbr.	Defined by	What the definition fixes
Autonomy Covenant	AC	Def. 3.1	$\mathcal{E} = E_1 \wedge E_2 \wedge E_3 \wedge E_4$. A conjunction, not a weighted score: no narrow σ compensates for $\rho = \varepsilon$
Model-Excluded Authority	MEA	Def. 3.2, Prop. 3.4, Cor. 3.6	The domain of δ contains no learned output. Independence is a property of the <i>signature</i> , surviving retraining and reprompting
Action Contract Object	ACO	Def. 3.1 (σ, ρ declared), Prop. 3.5	The action is declared in full before execution, and drawn from a closed catalogue $\mathcal{C}$
Evidence-Bound Targeting	EBT	Def. 3.7, Req. 3.8, Def. 3.9	ext has structured evidence as its domain. Free text cannot select a victim, because it is not in the domain
Principal-Perimeter Binding	PPB	E_1 , §5.1	$\text{ep}(a) \in \Pi(p)$ by strict equality, and the writing credential confined to $\Pi(p)$. No default perimeter
Discriminator-Scoped Containment	DSC	Def. 5.4, Thm. 5.5, Cor. 5.7	Per-principal containment is expressible iff a separating predicate exists per principal. Where none exists, the principal is refused rather than recorded as defended
Split-Authority Verification	SAV	E_4	The postcondition is read through a credential with no write capability. Disjointness of capability, not of convention
Sealed Session Chain	SSC	§4.2, Def. 4.1, Thm. 4.3	Digest over a fixed ordered field set including the predecessor's; terminal digest sealed under separate write authority
ProofPath Receipt	PPR	Def. 4.6, Req. 4.7, Cor. 4.4	The receipt carries enough that a verifier importing none of the emitter's code recomputes the chain — and localizes an alteration exactly
Element-Level Reconciliation	ELR	Prop. 6.3, Cor. 6.4, §6.3	Under aggregation the origin map is not recoverable, so reconciliation keys on the contained element, never on the provider's rule
Review-Bound Testimony	RBT	§6.6	Human evidence enters the record and is read by a deterministic verdict. It never commands
Sealed Review Loop	SRL	§6.5, Def. 6.9	Post-action review is a decision function over (present, recurred, elapsed), recorded in a <i>new</i> sealed session that cites the original without reopening it

Why naming is load-bearing here, and not decoration

Three reasons, and the third is the one that matters.

A conjunction needs a name for its own sake. $E_1 \wedge E_2 \wedge E_3 \wedge E_4$ is one object with one failure mode — any conjunct false, the action is inadmissible — and a prose that spells out four predicates each time invites the reader to treat them as a checklist from which one might be traded away. *AC* is the conjunction; there is no partial AC.

The pairs that are easy to confuse are easy to separate once named. SAV and PPB both concern credentials and are different properties: PPB is about *where* the writing credential may write, SAV about a *second* credential that cannot write at all. A system can satisfy PPB and fail SAV — it confines its writes and then believes its own report. §9's F1 is precisely that failure.

A named mechanism can be checked for existence; an unnamed one is assumed into being. This is the reason. A framework paper can define twelve mechanisms and an implementation can carry eleven, and the gap is invisible while the twelfth is discussed in prose that everyone reads as description. Naming makes the inventory finite and therefore auditable: one can ask, of a system claiming to be an ADAI, which of the twelve it implements — and get an answer that is wrong when it is wrong.

The three that are not properties of a single episode

ELR, RBT and SRL do not hold or fail *within* an episode, and treating them as if they did would manufacture a verdict where there is none.

ELR is a property of a reconciliation pass over the set of effects in force, not of any one containment. **RBT** applies only where testimony exists, and most episodes have none. **SRL** closes in a *different* session from the one being reviewed — by construction, since reopening the original would violate SSC.

A conformance report over an episode must therefore distinguish *did not hold* from *does not apply here* and from *not decidable from this record*. Collapsing the latter two into the first manufactures failures; collapsing them into success manufactures assurance. The second error is the dangerous one, and it is the one a reader wants by default.

8. Evaluation methodology

We distinguish three families. Only the first is a machine-learning evaluation, and reporting only the first — which is the field's prevailing practice — leaves the binding deployment constraints unmeasured.

8.1 Policy quality

Given a signal, does the system propose the correct action? Measured against a blind holdout of real sessions with **verified** outcomes, where "correct" is defined by the verified outcome rather than by an annotator's judgement.

Standard metrics apply, with one addition. Define the **harm-weighted error**

$$\mathcal{H} = \frac{1}{N} \sum_{i=1}^N w(\sigma_i) \cdot \mathbb{1}[a_i \neq a_i^*], \quad w(\sigma) = B(\sigma) (\Delta_{\text{det}} + \gamma (1 - \text{rev}(\sigma))),$$

with $\text{rev} \in [0, 1]$ a reversibility score, Δ_{det} the expected time from the erroneous action to its reversal, and $\gamma > 0$ an irreversibility premium. γ carries units of time, so that w has units of *principal-seconds of wrongful denial* — a quantity with an operational meaning, unlike a dimensionless score.

The additive form is deliberate. The tempting weight $B(\sigma)(1 - \text{rev}(\sigma))$ assigns **zero** cost to a fully reversible error, which is false: a reversible wrong block still denies service for the interval between the error and its reversal, and that interval is Δ_{det} — a property of the review loop, not of the action. Reversibility bounds the *tail* of the damage; it does not eliminate the *head*. A metric that cannot see the head rewards a system that reverses quickly and errs constantly.

Unweighted accuracy, in turn, treats a wrongly-challenged single address and a wrongly-blocked provider as one error each, which is precisely the conflation §1.3 identifies.

The holdout must be adversarial by construction: it must contain signals crafted to elicit a wrong action, including ones whose free-text fields name innocent third parties (testing Req. 3.8), and the benefit of the doubt must never be given.

8.2 Envelope integrity

Does the system refuse what the envelope must refuse? This is not measured on a sample; it is measured by adversarial construction. For each predicate E_i , construct actions violating E_i and confirm refusal:

$$\text{EIR} = \frac{\#\{\text{constructed violations refused}\}}{\#\{\text{constructed violations}\}}.$$

The target is $\text{EIR} = 1$ exactly, because $\mathcal{E}$ is a conjunction: a single admitted violation means the predicate is not enforced, and a rate of 0.99 describes a system with no guarantee rather than a nearly-guaranteed one.

8.3 Evidence integrity

Three quantities:

- **Recomputation rate:** fraction of exported receipts a third-party verifier (Req. 4.7) validates.
- **Tamper detection rate:** fraction of synthetically altered chains rejected.
- **Tamper-localization accuracy:** $\Pr[v(T') = j^*]$, by Cor. 4.4 expected to be 1; a value below 1 indicates the chain is functioning as a checksum rather than as a chain.

8.4 What cannot be measured, and must be constructed

Certain properties admit no finite test, because a test would have to enumerate an open set:

- no credential can write outside its perimeter;
- no learned component can widen the permitted set;
- no path exists from free text to an executed action.

Each is a universally quantified statement over an unbounded space. They are discharged **by construction** — respectively by credential scoping in the sense of least privilege [32, 9, 23] (Def. 3.1 E_1), by the signature of δ (Prop. 3.4), and by the domain of ext (Req. 3.8) — and the corresponding tests assert the *construction*, not the absence of counterexamples.

This position — that safety properties of learning systems must be discharged architecturally rather than statistically — is argued in the machine-learning security literature [2, 28]; we give it a concrete instance.

This is the paper's central methodological claim. For an agent acting on production infrastructure, the properties that matter most admit no test, and the architecture must therefore locate them where testing is unnecessary. A design in which these properties are testable is a design in which they are contingent.

8.5 What this paper measures, and what it does not

We report none of §8.1. The deployment of §9 has produced too few outcome-verified episodes to construct a holdout, and the adversarially constructed holdout that family requires — signals crafted to elicit a wrong action — is specified there and not built. Note that this is a different artifact from the envelope harness below: one attacks the *choice*, the other attacks the *permission*, and building the second

establishes nothing about the first. A reader should therefore take §§3–7 as an architecture with proofs and §9 as a set of counterexamples, and should not take anything in this paper as evidence that the resulting agent *chooses well*. That question is open and we are not in a position to close it.

We report §8.2. Twelve violations were constructed across the four predicates — three or more per predicate — and all twelve were refused, giving $EIR = 12/12 = 1$. The construction includes the target-provenance attack of Req. 3.8 (an address named only in an adversary-influenced free-text field), an action declaring effect scope outside the defended perimeter, an action at maximum autonomy with an empty rollback, and a catalogue-widening attempt in which a proposed action name does not exist in $\mathcal{C}$.

We report §8.3, over an exported receipt of a twelve-turn production session, recomputed by a verifier that imports no part of the emitting system and takes the hash function, the ordered field list, the serialization rules and the chaining rule from the receipt itself. Recomputation rate 1; tamper detection rate 1 over 36 constructed alterations — every turn index under each of the three adversary strategies of Thm. 4.3; localization accuracy 1 over the 23 alterations at which some conjunct failed.

The remaining 13 are reported separately rather than folded into either rate, because folding them in either direction misstates the property. They are the cases in which **no conjunct fails by construction**: the rewritten suffix, and alteration of the *final* turn with its digest recomputed, which has no successor link to carry the discontinuity. In both, detection falls to the separately stored seal, and there is no index to localize. Counting them as localization failures would understate a chain that behaved exactly as Thm. 4.3 predicts; silently excluding them would hide the boundary at which the chain stops localizing and starts merely detecting.

These two families are measurable precisely because they are properties of the construction rather than of a sample, which is also why the omission of §8.1 is statable: the arguments of §§3–7 do not become weaker because policy quality is unmeasured. A framework whose safety argument depended on the model's accuracy would not survive that admission, and §1.3 is the reason it does not depend on it.

One caveat belongs with the EIR figure and not in a footnote. $EIR = 1$ licenses the claim that *the constructed violations are refused*. It does not license "the envelope is inviolable" — that is a universally quantified claim over an open set, and §8.4 is the reason no harness establishes it. Enlarging what the number licenses requires adding constructions, not repeating the measurement.

8.6 What a passing result licenses

Less than it appears. Policy quality licenses a claim about the holdout's distribution. Envelope integrity licenses a claim about the predicates attacked. Evidence integrity licenses a claim about the artifact, not about the world it describes.

None licenses "safe to run autonomously". That rests on the conjunction of the envelope holding, the review loop closing, and the human retaining reversal — and the third is an operational commitment, not a measurement.

9. Observed failure classes

The following were observed in a production deployment built to this framework, defending three services behind one shared enforcement domain.

The evidential status of this section is that of a counterexample, not of a sample. Each item establishes that a failure class is *reachable* in a real system with a real test suite — an existence claim, which one instance settles and which no amount of additional instances would strengthen. We make no frequency claim, and none of the arguments in §§3–6 rests on one. What the section is for is to rule out the reading that these classes are hypothetical: the objection "a careful team would not do that" is answered by a careful team having done it, with tests, and not noticing.

Each is *silent* absent the corresponding structural guarantee, and each was invisible to the tests in place.

F1 — Isolation verified on the read path, assumed on the action path. Tenant filtering was complete and tested in every query; the action path resolved its perimeter from process configuration rather than from the session's principal. The failure is unreachable while one principal produces signal and becomes reachable the instant a second is onboarded. *Class*: a guarantee audited where auditing is cheap and assumed where it is expensive. *Discharged by*: E_1 with strict equality and no default — the default "if unknown, use the house perimeter" is exactly how one principal's incident becomes another's outage.

F2 — A receipt that no one could recompute. The receipt exported a display projection carrying 5 of the $k = 10$ hashed fields. Recomputation per the receipt's own instructions failed on every turn. Every internal test passed, because every internal test constructed the turn shape through the producer's own code. *Class*: a property of an artifact, tested through its producer. *Discharged by*: Def. 4.6(vi) and Req. 4.7.

F3 — A query for a value the writer never writes. The review query selected turns whose kind literal differed from the one persisted. The join returned the empty set, silently, for weeks: executed containments were never reviewed, and the guard against re-adoption — using the same literal — was permanently false, producing Cor. 6.4's false-positive mode at hourly cadence. No test caught it, because all used a storage double, and a double returns what the test specifies. *Class*: a defect in the interface between two individually-correct, individually-tested components — a known hazard of test doubles, which encode the implementer's belief about the other side rather than the other side [22]. *Discharged by*: deriving the value from one declaration at both ends, making mismatch a load-time error rather than an empty result.

F4 — A documented guarantee that was never enforced. A design record asserted that a build gate prevented third-party model dependencies in the reasoning path. The gate checked something else. The absence was in fact true — by discipline, not by enforcement. *Class*: documentation that switches off inspection; a reader who believes "the gate covers this" stops checking, and the cost materializes not when the statement becomes false but when someone relies on it. *Discharged by*: the process rule that every asserted guarantee carries an enforcing check or states in its own text that it is discipline.

F5 — Field-name divergence across an ingestion boundary. Two access paths to the same provider's events use different field capitalizations. The reader used one; the delivery path produced the other. Every event lost the adversary's address, silently — and the test passed, because its double mixed both spellings, a shape the provider never emits. *Class*: a test double encoding the implementer's belief about an external format rather than the format. *Discharged by*: fixtures captured verbatim from the external system.

F6 — Scheduled components running stale code. The deployment mechanism updated long-running services and not scheduled tasks, each of which remained pinned to whatever artifact a human last registered. The task closing the containment loop, and the task reading the threat feed, both lagged the tested tree — so F5's fix was merged, tested, deployed, and never reached the component that needed it. *Class*: divergence between the tested artifact and the running artifact along a path no test observes. *Discharged by*: one deployment mechanism for every component that runs, reporting what it changed.

F7 — A predicate whose implementation was narrower than its statement. E_2 requires $\text{effect}(a) \subseteq \sigma$ with σ bounded by $\Sigma_{\max}$, and the implementation enforced it by rejecting targets that are private, loopback, link-local, multicast or reserved — the addresses that are not routable sources. It admitted addresses belonging to the *enforcement point's own published ranges*. Two containments against the enforcement provider's shared proxy addresses stood in a production zone until they were found by inspection.

The instructive part is what did **not** go wrong. The evidence was structured, not free text; the extraction was sound; the model proposed containment of the address the evidence named, which was the correct address from the evidence's point of view — under a proxied path, the observed source is the proxy. Nothing in the reasoning path erred. The action was nonetheless outside E_2 : a shared proxy address has $B(\sigma) \gg 1$, since the effect falls on every principal whose traffic arrives through it, and among them the defended principal itself. It is additionally the path by which the containment would be observed, so E_4 's verification and the effect become mutually dependent.

Class: a predicate stated correctly and implemented as a proper subset of itself — the gap invisible because the implemented subset is the one a reader checks for. It is the mirror image of F4: there, documentation asserted a check that did not exist; here, a check existed and was quietly weaker than the predicate it was taken to discharge. *Discharged by*: enumerating the enforcement point's published ranges as static, dated data on the decision path — static because resolving them at decision time would make δ depend on a network service and cease to be deterministic, which would break the recomputation contract of Def. 4.6 — and by adding the case to the §8.2 harness, where its absence would otherwise have been invisible again.

Common structure. In all seven, the guarantee resided in convention rather than in construction, and the symptom was absence of signal rather than presence of error. F7 sharpens the point: the guarantee can reside in construction and still fail, if the construction implements less than the guarantee states — which is why §8.2 measures the predicates by attacking them rather than by reading the code that implements them.

This is the empirical case for the framework: not that structure prevents mistakes, but that it converts silent mistakes into loud ones.

10. Relation to prior work

Security orchestration. Complementary and subsumed [4, 17]. A reviewed playbook catalogue is exactly the right shape for the permitted set; Prop. 3.5 retains that property deliberately. What the framework adds is a principled account of when an agent may select from the catalogue unaided, and what it owes afterwards.

Autonomic computing. The framework is the MAPE-K loop [18, 16] with (i) a mandatory gate between Plan and Execute whose domain excludes the Knowledge base's learned contents, and (ii) a mandatory return path from Execute to Monitor with the world→records direction of §6. Both lie in the space MAPE-K leaves open.

Learned cyber defense. §8.1 is this literature's standard evaluation [37, 14, 25], adopted unchanged. Our claim is that it is one family of three and is not the binding constraint on deployment.

Tamper-evident logging. §4.2's construction is standard [13, 21, 35, 6, 20]. The contributions are Def. 4.6's recomputation contract, the observation that clause (vi) is routinely claimed and routinely unmet (F2), and Cor. 4.4's exact localization as a *measurable* rather than merely asserted property.

Capability security and separation of duties. Separation of duties is due to Clark and Wilson [5]; capabilities to Dennis and Van Horn [9], with the modern treatment in Miller [23]. E_4 is separation of duties realized as capability disjointness rather than as role assignment [33]. The distinction is that role assignment is a property of the program — which a refactor breaks silently, leaving verification that still returns success and proves nothing — whereas capability disjointness is a property of the credential, which a refactor cannot alter.

Multi-tenancy. The isolation literature concentrates on data-plane and storage isolation [19]. §5 concerns isolation of *effects on shared infrastructure*, where the shared resource is the enforcement point itself, and Thm. 5.5 gives the expressibility condition. We are not aware of an equivalent treatment.

11. The difficulties, resolved

An earlier draft of this paper closed with a list of open problems. That list is this section, and nothing in it is open. Each difficulty is stated as a condition with a construction, and where the condition fails the framework prescribes what the system must refuse to claim — which is itself a result, and the one this paper returns to most often.

We set them out here rather than leaving them as an agenda, because a framework that names a difficulty and does not return to it has named a gap in itself. What survives is measurement rather than method, and §11.5 says which measurements and why they are not the same thing as an open problem.

11.1 Reconciliation against a world of foreign granularity

§6 solved the case where effects aggregate into rules. The general problem is: when can records be reconciled against a world whose units of observation are not the system's units of effect?

Definition 11.1 (Atomization). Let O be the observable state of an enforcement point and $\mathcal{F}$ the set of effects the system can create. An *atomization* is a computable map $\alpha : O \rightarrow 2^{\mathcal{A}}$ into a set of *atoms* together with a map $\iota : \mathcal{A} \rightarrow \mathcal{F} \cup \{\perp\}$, such that (i) each atom is derivable from the observation alone, without appeal to the system's records; and (ii) ι is injective on atoms with $\iota \neq \perp$ — each atom corresponds to at most one effect the system could have created.

Theorem 11.2 (Reconciliation condition). World-to-records reconciliation is sound at an enforcement point iff an atomization exists. Under one, the algorithm is: compute $\alpha(O)$, and for each atom with $\iota(\text{atom}) \neq \perp$, test the records for a corresponding effect; atoms failing the test are orphans.

Proof. ($\Leftarrow$) Clause (i) makes the traversal computable from the world side, which is what record-driven review lacks. Clause (ii) makes the test well-posed: an atom maps to at most one effect, so "is this effect recorded?" has a determinate answer. Completeness follows because every effect the system created is present in O and therefore yields an atom. ($\Rightarrow$) Suppose reconciliation is sound. Then for each observation there is a procedure deciding, per created effect, whether it is recorded. The partition of O induced by that procedure's decisions is an atomization: it is computed from O alone, and its classes correspond to at most one effect each, since otherwise two effects would share a decision and one could be recorded and the other not. $\square$

Corollary 11.3 (Where reconciliation is impossible, and what to do). If the enforcement point exposes only aggregate state from which individual effects are not recoverable — a counter, a rate, a boolean "protection enabled" — then no atomization exists, and reconciliation is impossible rather than merely unimplemented.

The correct response is the one §5.2 already prescribes for undefendable principals: refuse the capability rather than claim it. A system integrating such an enforcement point must record that its effects there are **unreconcilable**, and must not report a reconciliation pass over them as clean. Reporting "0 orphans" when orphans are undetectable is the most damaging sentence such a system can emit, because it is indistinguishable from the true one.

The three layouts of §5.3 are now instances. Per containment, α is the identity on rules and ι reads the rule's description. Per principal, α decomposes the rule into its address set and ι maps each address to the containment of that address for that principal — which is precisely why reconciliation had to move to the element (Prop. 6.3). Under the single shared rule, α must recover both address and principal from one expression; it can, so reconciliation remains possible, which is worth noting because it means the shared layout is rejected for the reasons in §5.3 and not for this one.

11.2 Cross-perimeter correlation without cross-perimeter disclosure

An adversary probing q principals is one campaign. The evidence that would reveal it is, by E_1 , partitioned per principal. The apparent dilemma is that correlation requires comparing evidence across the partition.

It does not. Correlation requires comparing *one derived value* per observation, and the value can be chosen to carry no other information.

Construction 11.4 (Blinded correlation index). Fix a key κ held by the installation and not by any principal. On each containment of target x for principal p , the session emits, into a shared append-only index outside every principal's partition, the tuple

$$\langle \text{PRF}_{\kappa}(x), \text{timestamp}, \text{signal type} \rangle$$

and **no principal identifier**. Correlation is then the computation of multiplicities over the index: a value appearing k times with $k > 1$ indicates one source contained in k perimeters.

Proposition 11.5 (What the index does and does not hide). For a reader holding neither κ nor the index's preimages, the index reveals the *number* of perimeters a source reached and the times, and reveals neither the source nor the principals. For a reader holding κ , it reveals the source: the address space is small enough to enumerate, so PRF_{κ} is not hiding against a key-holder.

Proof. Without κ , $\text{PRF}_{\kappa}(x)$ is indistinguishable from random by the PRF assumption, and the tuple carries no principal field, so no principal is identifiable. With κ , computing $\text{PRF}_{\kappa}(x')$ for all x' in the address space recovers x by table lookup; for IPv4 this is 2^{32} evaluations, which is not a barrier. $\square$

This bounds the construction precisely, and the bound is the useful part: the index protects one principal's evidence **from another principal**, which is the property E_1 is about, and does not protect it from the operator, who already holds every perimeter's credentials and therefore already sees every containment. Deploying the index with κ held by the operator is therefore not a weakening of E_1 — it grants the operator nothing it lacked — while giving principals an aggregate signal they could not otherwise obtain.

Two properties keep the construction inside the framework. The index is **append-only and outside every partition**, so a write to it is not a cross-tenant read. And it carries **no principal identifier**, so even a key-holder learns "this source hit k perimeters" rather than which — a distinction that matters if the index is

ever disclosed, and that costs nothing to preserve.

The residual limitation is honest and narrow: if q is small, the count itself is identifying, since "3 of 3 perimeters" names them. Below a threshold on q the construction should not be deployed, and the threshold is a deployment decision rather than a cryptographic one.

11.3 Corpus scarcity: the arithmetic is better than first stated

We wrote that the learning signal is sessions with verified outcomes, that the bar is high, and that few examples accrue per deployment. The first two clauses are right and the third understates the corpus by conflating *containments* with *labelled examples*.

Each containment yields a sequence of labels, not one. The review loop (§6.5) revisits every containment on a schedule and emits a verdict each time: reaffirmed, too early, expired, absent, or false positive. A containment that lives T hours under an hourly review contributes on the order of T labelled observations of the same decision under changing evidence — and the *transitions* between verdicts are the most informative examples in the corpus, being exactly the moments at which the world contradicted an earlier judgement.

Refusals are outcome-verified data and are far more abundant than containments. Every signal that curation rejects, every extraction that yields no actionable target, every signal type absent from the catalogue, and every denial by the decision function is a recorded episode with a determinate outcome and a written reason.

This was asserted in an earlier draft and is now measured. Over a thirty-day window in the deployment of §9, with all identifiers omitted:

Quantity	Count
Episodes in the operator's tenant	114
— terminated without action	81
— sealed	33
Refusal turns, 30-day window	93
— attributable to a single criterion (severity below the materiality threshold)	79 (85%)
Signals whose evidence never yielded an actionable target	10
Containments with a reviewed outcome	4

Two things in this table matter more than the ratio. The first is that refusals exceed containments by roughly an order of magnitude, which settles the arithmetic question and makes the training-set composition the opposite of what one would guess. The second is the 85%: a *single* criterion accounts for most of what the system declines to look at, which means that criterion, alone, defines the system's field of view. That is a finding about the deployment rather than about the framework, and it is the kind of finding the measurement layer exists to surface — it was produced by the system's own findings layer, not by an analyst querying the database.

Training a model to narrate only what was contained would therefore discard about nine tenths of the corpus, and the more informative nine tenths: the boundary of the permitted set is drawn by the refusals, not by the actions.

Adoptions are labelled too. An adopted effect (§6.2) enters review and accumulates verdicts identically, despite having no execution receipt.

What remains genuinely open is narrower than "corpus scarcity", and we state it in its narrow form: whether the *distribution* of a corpus so composed — dominated by refusals, with containments as a minority class and verdict transitions rarer still — supports training a narrator that does not degrade on the minority class. That is a class-imbalance question with a substantial literature and no result specific to this framework, and it is an empirical question this paper does not settle.

11.4 Composite actions across enforcement domains

$\mathcal{E}$ is stated for a single perimeter. An action spanning two domains with different expressible predicates and different credentials requires a composition rule, and the difficulty is E_3 : the rollback of a composite is not in general the composition of the rollbacks.

Let a **composite action** be an ordered sequence $a = \langle a_1, \dots, a_n \rangle$ with a_i acting at domain Z_i , each a_i carrying its own declared rollback ρ_i and effect scope σ_i .

Theorem 11.6 (Composite reversibility). Suppose each a_i satisfies E_3 individually, and the effect scopes are pairwise disjoint: $\sigma_i \cap \sigma_j = \emptyset$ for $i \neq j$. Then the composite satisfies E_3 with rollback $\rho = \langle \rho_n, \dots, \rho_1 \rangle$, and moreover ρ restores the identity on $\bigcup_i \sigma_i$ regardless of which prefix of a executed.

Proof. Disjointness makes the $\text{exec}(a_i)$ pairwise commuting on $\bigcup_i \sigma_i$, since each acts as the identity outside its own scope. Each $\text{exec}(\rho_i) \circ \text{exec}(a_i) = \text{id} \upharpoonright_{\sigma_i}$ by hypothesis. For a prefix $a_1 \dots a_m$, applying $\rho_m, \dots, \rho_1$ gives id on $\sigma_1 \cup \dots \cup \sigma_m$ and, by disjointness, leaves $\sigma_{m+1} \dots \sigma_n$ untouched — where nothing was done. Reverse order is not required by commutativity but is required by E_3 's recording discipline, so that a compensation never precedes the action it compensates in the transcript. $\square$

Corollary 11.7 (Refusal condition). If $\sigma_i \cap \sigma_j \neq \emptyset$ for some $i \neq j$, the composite does not in general satisfy E_3 and must be refused at the decision point rather than attempted with a best-effort rollback.

Justification. With overlapping scopes, ρ_j may restore a state of the overlap that a_i had deliberately changed, so the composition is not the identity; which of the two prevails depends on execution order, and a rollback whose result depends on interleaving is not a rollback. $\square$

Three consequences follow for implementation, and they are the practical content of the theorem.

Disjointness is checkable before execution, from the declared scopes, which is why E_2 requires the scope to be declared. The decision point therefore gains one rule: for a composite, verify pairwise disjointness of declared scopes and refuse otherwise.

The full compensation chain is recorded before the first execution, as E_3 already requires per action; the composite's recording is the concatenation, and it must be durable before a_1 runs. A crash at any point leaves a transcript from which the remaining compensations are computable.

Verification composes without a new rule. Each a_i is verified at Z_i by that domain's read-only credential, and the composite is verified iff every component is. No cross-domain verifier is needed, which matters because a cross-domain verifier would need write-capable reach into several domains — the opposite of E_4 .

11.5 Class imbalance: bounded by construction, not by hope

§11.3 reduced "corpus scarcity" to a narrower question: whether a corpus dominated by refusals trains a narrator that does not degrade on the minority classes — containments and verdict transitions. Left there, it is an empirical unknown. It need not be left there, because the architecture already forbids the bad outcome from reaching production.

The construction requires two things of the deployment, and we state them as hypotheses rather than citing them, because neither has been established earlier in this paper: **(G1)** publication of a trained narrator is gated on a measurement taken *per outcome class* against a deterministic baseline, publication being refused if any class falls below it; and **(G2)** in the absence of a published model, narration falls back to that deterministic rule.

Proposition 11.8 (Degradation cannot reach production). Under G1 and G2, for any training distribution, the deployed narrator is either at least as good as the deterministic rule on every class, or there is no deployed narrator and the system narrates by rule.

Proof. By G1 the gate is a precondition on publication and its measurement is class-conditional, so a model degrading on any class fails it and is not published. By G2 the remaining branch narrates by the deterministic rule, whose behaviour is independent of the corpus and therefore of its class distribution. Both branches satisfy the claim. $\square$

G1 and G2 are properties a deployment either has or does not, and the companion paper reports a system that has both.

The proposition is not a claim that the model will be good. It is a claim that the **risk is one-sided**: class imbalance can cost the deployment a narrator, and cannot cost it a wrong narration. That asymmetry is what makes the question non-blocking, and it exists because the narrator does not decide (criterion C) — a learned component excluded from the admissibility decision can fail without the failure propagating into an effect.

Two constructions improve the odds rather than merely bounding the loss.

Harm-weighted stratification. Training examples are weighted by the operational cost $w(\sigma)$ of §8.1 rather than by frequency. The weight already exists, is already defined in units with operational meaning, and encodes exactly the judgement that a wrong containment matters more than a wrong refusal. Using frequency instead would optimize for the class the system cares least about.

Transitions as a class of their own. §11.3 observed that verdict transitions are the most informative examples. They are also the rarest, so under frequency weighting they vanish. Treating them as a distinct outcome class, with its own gate threshold, prevents a model from passing by performing well on the steady state and badly at exactly the moments the record exists to capture.

What remains is a measurement, not a problem: the class-conditional numbers, once a corpus of sufficient size exists. §8.5 already states that this paper reports no policy-quality measurement, and the reason is the same.

11.6 Enforcement points that do not atomize: making the failure loud

Cor. 11.3 showed that an enforcement point exposing only aggregate state admits no atomization, and prescribed refusing the reconciliation capability. Refusal is correct and incomplete: it leaves the system unable to defend such a point at all, if defending it honestly requires reconciliation.

Two constructions close this, and they trade against each other.

Construction 11.9 (Atomize by refusing aggregation). Where the enforcement point exposes per-object state but the system had aggregated several effects into one object, the atomization is recovered by allocating one enforcement object per effect — the per-containment layout of §5.3. Reconciliation is then rule-identified and sound.

This is exact and costs capacity: $\mathcal{R} = N$ against a budget of $L - k$ (Prop. 5.9). It is the right choice wherever the budget permits, and §5.3's argument for aggregation applies only where it does not.

Construction 11.10 (Write-ahead intent log). Where the enforcement point exposes no per-effect state at all — a counter, a rate, a boolean — no allocation recovers an atomization, because the deficiency is in O and not in the system's use of it. Introduce instead a second store I , under a write authority distinct from both the enforcement point and the transcript. Before $\text{exec}(a)$, the system appends to I an *intent* record naming the enforcement point, the intended effect, and the session. After a verified compensation, the intent is marked discharged.

Reconciliation then runs over I rather than over O : an undischarged intent with no corresponding session record is an orphan.

Proposition 11.11 (What the intent log recovers, and what it does not). With I , every effect the system attempted at a non-atomizable enforcement point is detectable from I alone. An undischarged intent does **not** establish that the effect took hold, since O cannot be queried per effect; it establishes that the system tried and that no record of the outcome exists.

Proof. The intent is written before execution and is therefore present for every attempt, including those whose session write failed — which is the window §6.1 identifies. Conversely, I records intent and not observation, so it cannot distinguish an effect that took hold from one whose execution failed at the enforcement point, that distinction requiring exactly the per-effect query that O does not support. $\square$

The gain is precisely the one this framework pursues everywhere: it converts an **undetectable** orphan into a **detectable ambiguity**. Before, the system could say nothing and did not know it was saying nothing. After, it can say "an effect was attempted here, its outcome is unrecorded, and this enforcement point cannot tell me which" — a sentence an operator can act on, and one the system is obliged to emit rather than reporting a clean pass.

Corollary 11.12 (The reporting rule). A system integrating a non-atomizable enforcement point reports three counts and not one: reconciled, orphaned, and **unverifiable**. Collapsing the third into either of the first two is the error Cor. 11.3 warns about, and the intent log is what makes the third count computable instead of merely conceivable.

Together, Constructions 11.9 and 11.10 leave no enforcement point at which the system must either lie or abstain. Where the budget permits, atomize exactly. Where it does not and O is per-object, aggregate and reconcile per element (§6.3). Where O is aggregate, use the intent log and report the third count.

12. Conclusion: autonomous defense is possible, and this is how

The question this paper set out to answer was whether a machine may be permitted to defend infrastructure on its own — deciding, at the moment of an incident, without a human in the loop, and acting on systems that carry other people's traffic. We have argued that the answer is yes, that it is yes under stated conditions rather than in general, and that the conditions are structural rather than a matter of how good the reasoning component happens to be.

The results of §§3–11 assemble into a single constructive statement, and we give it in that form because a framework of prohibitions read as a list of prohibitions misrepresents what it is for.

Theorem 12.1 (Feasibility of autonomous defensive action). Let S be a system satisfying the following, each established by a preceding section:

1. **Evidence is grounded.** Targets are extracted only by a total parsing function over a fixed key set of the signal's structured evidence, excluding adversary-authored natural language (Req. 3.8).
2. **The permitted set is fixed before the reasoning component runs.** Actions are instantiated from a reviewed catalogue, and the admissibility function's domain excludes every learned output (Prop. 3.4).
3. **Every action lies in the envelope.** Perimeter bound to principal with no default, effect scope declared and bounded by a policy constant, rollback recorded before execution, and postcondition verified through a capability-disjoint credential (Def. 3.1).
4. **Per-principal containment is expressible.** The enforcement point admits a separating predicate for each defended principal (Thm. 5.5); principals for which it does not are refused registration (Cor. 5.7).
5. **Every effect is reachable from the records.** An atomization exists, or a write-ahead intent log supplies one (Thm. 11.2, Constr. 11.10).
6. **Every action is recorded verifiably.** A hash-linked transcript sealed under a separate write authority, exported with a recomputation contract (Def. 4.6).

Then S may act without contemporaneous human approval with the following guarantees, all of which hold independently of the quality of any learned component:

- **(a) Bounded harm.** $\max B(a)$ over every action S can execute, under every model behaviour, is a deployment constant computable without running the system (Cor. 3.4a).
- **(b) No cross-principal effect.** Every containment matches within exactly one principal's traffic (Thm. 5.5), and a session whose principal has no registered perimeter does not act (§5.2).
- **(c) Recoverable state.** Every effect has a rollback recorded before it, and composite actions across domains are reversible exactly when their declared scopes are disjoint — a condition checked before execution (Thm. 11.6).
- **(d) No silent effect.** Every effect created is reachable from the records or reported as unverifiable; none is invisible (Thm. 11.2, Cor. 11.12).
- **(e) Third-party-checkable account.** Any party may verify the record without the system's code and localize any alteration exactly (Thm. 4.3, Req. 4.7).

Proof. Each guarantee is the cited result under the cited hypothesis; the conjunction holds because the hypotheses are independent — (1) constrains extraction, (2) construction, (3) admissibility, (4) expressibility, (5) observability, (6) recording — and no result's proof appeals to the negation of another. $\square$

What this licenses operationally

Read against the phases of incident handling [4], the theorem covers three of them and stops precisely where the deployment's reach stops, which is worth stating rather than blurring.

Containment is direct. Guarantees (a)–(c) are exactly what is needed to permit a machine to change an enforcement point unsupervised: the damage is bounded before the fact, it cannot reach another principal, and it can be withdrawn. This is the phase the framework addresses most completely.

Eradication is bounded by the enforcement point's reach and we do not overstate it. A system acting at a network edge removes an adversary's *access*, not its foothold: it ends the reach, and cannot clean a compromised host. What guarantee (d) adds is the part eradication actually requires and that most automation lacks — that the response be *complete*, with every effect accounted for. An adversary's access is not eradicated by a block that some later process silently reverted and nobody noticed. The review loop

(§6.5) is what makes the removal an ongoing verified fact rather than a one-time write, and the reconciliation traversal (§6.2, §11.6) is what makes "there are no forgotten effects" a checkable statement instead of a hope.

Prevention is the loop's second-order product. Each refused signal, absent playbook and non-actionable target is recorded with its reason (§3.1, §12 of the system paper), and those records are the measurement layer's input: the catalogue grows where the gaps were, under human review. Across perimeters, the blinded index (Constr. 11.4) makes one adversary's campaign visible as one campaign, without any principal's evidence leaving its partition. Neither is prediction; both are the ordinary way a defense improves, made computable.

Why the reasoning component's fallibility is not fatal

The move that makes all of this work is a separation, and it is worth stating plainly because it is the paper's central claim about *how* autonomy becomes admissible.

A defensive agent needs judgement: which signals matter, which target is the adversary, which response fits. Judgement of that kind is the province of components that are fallible, that are trained on data, and whose input is in part written by the adversary. Building a safety argument on such a component's accuracy is building it on the adversary's cooperation.

So the system does not. The **set** of admissible actions is fixed by the catalogue and the envelope before any reasoning runs, and is checkable without running it. The **choice within that set** is where judgement lives, and where a learned component may contribute. A model that is wrong picks a permitted action that a better model would not have picked; it does not pick an impermissible one, because the impermissible ones are not in the set it is choosing from.

This is what makes the difference between an autonomous system and an unsupervised one. The first has a boundary someone drew deliberately and can inspect; the second has whatever boundary its most fallible component happens to produce. The failures in §9 are of two shapes, and both are silent for the same reason: six where the guarantee lived in convention rather than in construction, and one where it lived in construction but the construction implemented less than the guarantee stated. The second shape is the harder one, and it is the reason §8.2 attacks the predicates instead of reading the code that implements them — a boundary one can inspect is worth drawing only if inspection is something other than re-reading one's own intention.

The claim, stated exactly

A machine can be permitted to prevent, contain and account for attacks on production infrastructure, on its own, when: it acts only on evidence it extracted structurally and verified independently; the boundary of what it may do was drawn before it ran and does not move; every effect it creates is bound to one principal, reversible, and reachable from an evidential record; and that record can be checked by someone who does not trust it.

Those conditions are not easy, and this paper's contribution is that they are *stated* — as theorems with proofs, with the boundaries where each fails and an instruction to refuse the capability there rather than claim it. A system meeting them fails loudly where it fails — fail-stop rather than fail-silent [34, 10] — and that, not infallibility, is what a machine acting on other people's infrastructure owes the people who depend on it.

Limitations

The failure classes of §9 come from a single production deployment over a period of days. They are instances, not statistics; their evidential value lies in the *class* of each failure and in the demonstrated invisibility of each to the tests in place, not in any frequency claim. Theorems 4.3 and 5.5 are unconditional given their stated assumptions; Props. 3.4 and 3.5 are architectural and hold only for systems constructed as specified. No claim is made that the framework is sufficient for safety, only that each of its components is necessary for the guarantee it names.

The measurements of §8.5 carry their own bounds. $\text{EIR} = 1$ is over twelve constructions, and a harness measures its own coverage: a predicate is established only to the extent that the attacks on it were imagined. The evidence-integrity figures are over a single receipt — a full census of its turn indices and adversary strategies, and therefore exact for that chain, but one chain — so they establish that the recomputation contract is *satisfiable and satisfied here*, not a rate over a population. Neither family speaks to §8.1, and a reader who reads $\text{EIR} = 1$ as evidence that the agent acts *well* has read the wrong quantity: it is evidence that it cannot act *outside*.

References

- [1] Alahmadi, B. A., Axon, L., Martinovic, I. (2022). *99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms*. Proceedings of the 31st USENIX Security Symposium.
- [2] Biggio, B., Roli, F. (2018). *Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning*. Pattern Recognition 84, 317–331.
- [3] Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. Communications of the ACM 59(5), 50–57.
- [4] Cichonski, P., Millar, T., Grance, T., Scarfone, K. (2012). *Computer Security Incident Handling Guide*. NIST Special Publication 800-61 Revision 2.
- [5] Clark, D. D., Wilson, D. R. (1987). *A Comparison of Commercial and Military Computer Security Policies*. IEEE Symposium on Security and Privacy, 184–194.
- [6] Crosby, S. A., Wallach, D. S. (2009). *Efficient Data Structures for Tamper-Evident Logging*. Proceedings of the 18th USENIX Security Symposium.
- [7] Cvach, M. (2012). *Monitor Alarm Fatigue: An Integrative Review*. Biomedical Instrumentation & Technology 46(4), 268–277.
- [8] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F. (2024). *AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents*. arXiv:2406.13352.
- [9] Dennis, J. B., Van Horn, E. C. (1966). *Programming Semantics for Multiprogrammed Computations*. Communications of the ACM 9(3), 143–155.
- [10] Gray, J. (1986). *Why Do Computers Stop and What Can Be Done About It?* Symposium on Reliability in Distributed Software and Database Systems, 3–12.
- [11] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M. (2023). *Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. 16th ACM Workshop on Artificial Intelligence and Security (AISec), 79–90.

- [12] Guo, C., Pleiss, G., Sun, Y., Weinberger, K. Q. (2017). *On Calibration of Modern Neural Networks*. Proceedings of the 34th International Conference on Machine Learning, 1321–1330.
- [13] Haber, S., Stornetta, W. S. (1991). *How to Time-Stamp a Digital Document*. Journal of Cryptology 3(2), 99–111.
- [14] Hammar, K., Stadler, R. (2020). *Finding Effective Security Strategies through Reinforcement Learning and Self-Play*. 16th International Conference on Network and Service Management (CNSM).
- [15] Hanappi, O., Hummer, W., Dustdar, S. (2016). *Asserting Reliable Convergence for Configuration Management Scripts*. OOPSLA, 328–343.
- [16] IBM (2006). *An Architectural Blueprint for Autonomic Computing*, 4th edition. IBM White Paper.
- [17] Islam, C., Babar, M. A., Nepal, S. (2019). *A Multi-Vocal Review of Security Orchestration*. ACM Computing Surveys 52(2), Article 37.
- [18] Kephart, J. O., Chess, D. M. (2003). *The Vision of Autonomic Computing*. IEEE Computer 36(1), 41–50.
- [19] Krebs, R., Momm, C., Kounev, S. (2012). *Architectural Concerns in Multi-Tenant SaaS Applications*. 2nd International Conference on Cloud Computing and Services Science (CLOSER), 426–431.
- [20] Laurie, B., Langley, A., Kasper, E. (2013). *Certificate Transparency*. RFC 6962, IETF.
- [21] Merkle, R. C. (1988). *A Digital Signature Based on a Conventional Encryption Function*. Advances in Cryptology — CRYPTO '87, LNCS 293, 369–378.
- [22] Meszaros, G. (2007). *xUnit Test Patterns: Refactoring Test Code*. Addison-Wesley.
- [23] Miller, M. S. (2006). *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. PhD thesis, Johns Hopkins University.
- [24] Moreau, L., Groth, P. (2013). *Provenance: An Introduction to PROV*. Morgan & Claypool.
- [25] Nguyen, T. T., Reddi, V. J. (2023). *Deep Reinforcement Learning for Cyber Security*. IEEE Transactions on Neural Networks and Learning Systems 34(8), 3779–3795.
- [26] OASIS (2013). *eXtensible Access Control Markup Language (XACML) Version 3.0*. OASIS Standard.
- [27] Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J. V., Lakshminarayanan, B., Snoek, J. (2019). *Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift*. Advances in Neural Information Processing Systems 32.
- [28] Papernot, N., McDaniel, P., Sinha, A., Wellman, M. P. (2018). *SoK: Security and Privacy in Machine Learning*. IEEE European Symposium on Security and Privacy, 399–414.
- [29] Parasuraman, R., Sheridan, T. B., Wickens, C. D. (2000). *A Model for Types and Levels of Human Interaction with Automation*. IEEE Transactions on Systems, Man, and Cybernetics — Part A 30(3), 286–297.
- [30] Perez, F., Ribeiro, I. (2022). *Ignore Previous Prompt: Attack Techniques for Language Models*. arXiv:2211.09527.
- [31] SAE International (2021). *J3016: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles*.
- [32] Saltzer, J. H., Schroeder, M. D. (1975). *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), 1278–1308.

- [33] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., Youman, C. E. (1996). *Role-Based Access Control Models*. IEEE Computer 29(2), 38–47.
- [34] Schlichting, R. D., Schneider, F. B. (1983). *Fail-Stop Processors: An Approach to Designing Fault-Tolerant Computing Systems*. ACM Transactions on Computer Systems 1(3), 222–238.
- [35] Schneier, B., Kelsey, J. (1999). *Secure Audit Logs to Support Computer Forensics*. ACM Transactions on Information and System Security 2(2), 159–176.
- [36] Sheridan, T. B., Verplank, W. L. (1978). *Human and Computer Control of Undersea Teleoperators*. Technical Report, MIT Man-Machine Systems Laboratory.
- [37] Standen, M., Lucas, M., Bowman, D., Richer, T. J., Kim, J., Marriott, D. (2021). *CybORG: A Gym for the Development of Autonomous Cyber Agents*. arXiv:2108.09118.
- [38] Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., Thomas, C. B. (2018). *MITRE ATT&CK: Design and Philosophy*. MITRE Technical Report MP180360.
-

Authorship

Hiago Kin Levi

Brooklyn, NY — 16 September 2026, 22:00

SHA-256 of the Markdown source, to the marker line below:

b04015f05b23afc4b5abcad10ef739ab918627ea9ce56a70efbcc94dfabea382